

Project 1: Pure-Linux Container Network (Mini-CNI)

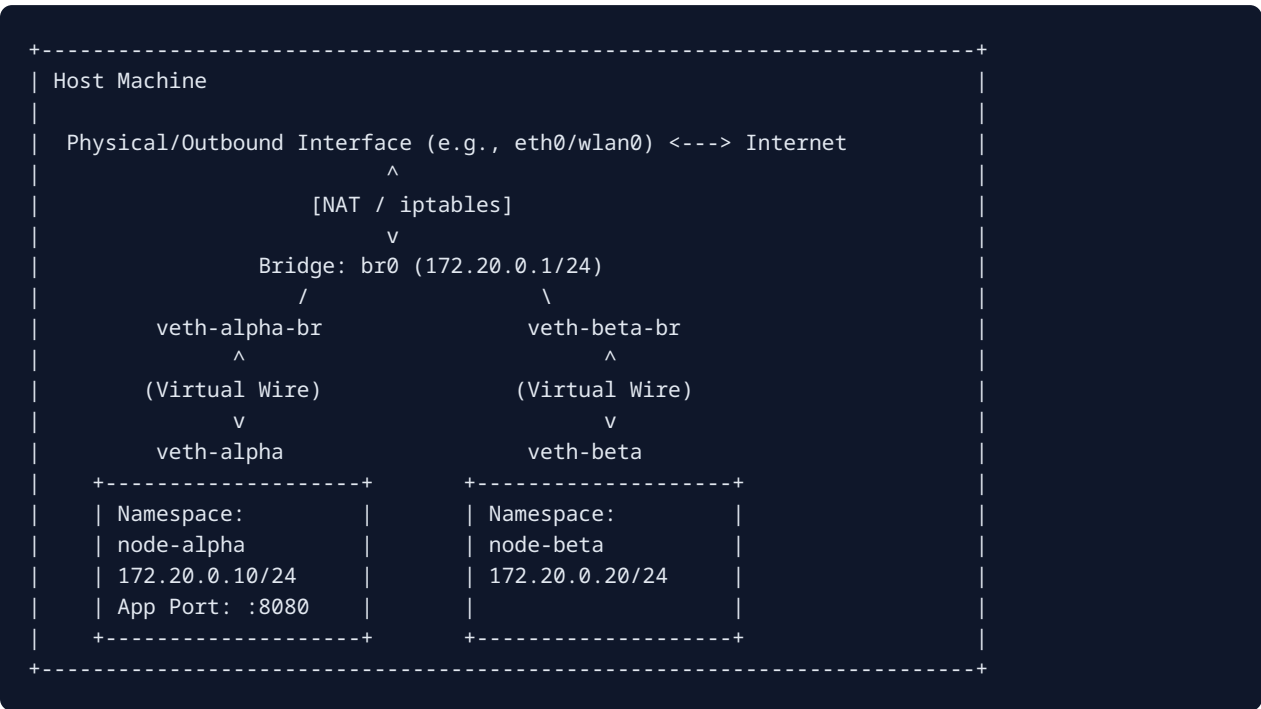
Specification & Engineering Brief • Native Kernel Primitives Implementation

1. Overview & Objective

Construct and document a fully operational virtual container network on a single Linux host from scratch. This project isolates and exercises the exact networking primitives that container runtimes (Docker, Podman) and Kubernetes CNI plugins rely on behind the scenes.

Strict Operational Constraint: No container runtimes (Docker, Podman, containerd, etc.) or external orchestration tools are permitted. All functionality must rely strictly on standard Linux kernel networking subsystems and userspace utilities.

2. Architectural Topology



3. Functional Requirements

Requirement 1: Namespace Isolation

- Create two distinct network namespaces: `node-alpha` and `node-beta`.
- Bring up the loopback (`lo`) interface within each namespace to allow local IPC and binding on `127.0.0.1`.
- Verify that both namespaces maintain isolated routing tables and network socket spaces.

Requirement 2: Virtual Bridge & Layer-3 Switching

- Create a virtual Linux bridge named `br0` on the host assigned `172.20.0.1/24`.
- Connect each namespace to `br0` via virtual Ethernet (`veth`) pairs.
- Assign static IP addressing:
 - `node-alpha`: `172.20.0.10/24`
 - `node-beta`: `172.20.0.20/24`
- Establish default gateway routes inside both namespaces pointing to the bridge gateway (`172.20.0.1`).
- **Acceptance Criteria:** `node-alpha` and `node-beta` can successfully exchange ICMP echo requests and establish TCP connections directly.

Requirement 3: Egress Internet Routing (SNAT / Masquerading)

- Enable IPv4 kernel packet forwarding (`net.ipv4.ip_forward = 1`).
- Configure an outbound NAT (Masquerading / SNAT) rule on the host's primary internet-facing interface.
- Adjust firewall forward chains to permit packet flow between the bridge subnet and external network interfaces.
- **Acceptance Criteria:** Processes inside both namespaces can ping external public IPs (e.g., `1.1.1.1` or `8.8.8.8`).

Requirement 4: Ingress Port Forwarding (DNAT)

- Run a lightweight HTTP daemon inside `node-alpha` listening on `0.0.0.0:8080`.
- Implement an inbound DNAT rule on the host that intercepts traffic sent to `<Host-IP>:9090` and forwards it directly to `172.20.0.10:8080`.
- **Acceptance Criteria:** Sending an HTTP request to `<Host-IP>:9090` returns the HTTP response generated by `node-alpha`.

Requirement 5: Isolated DNS Resolution

- Configure name resolution inside both namespaces without bind-mounting the host's default `/etc/resolv.conf`.
- Ensure UDP port 53 traffic routes across the bridge to upstream public resolvers.
- **Acceptance Criteria:** Resolving domain names (e.g., `curl -I https://google.com`) succeeds from within each namespace.

4. Deliverables Expected for Review

Artifact	Description	Validation Standard
<code>setup.sh</code>	Idempotent provisioning script for full topology setup.	Zero manual steps required; cleanly repeatable.
<code>cleanup.sh</code>	Teardown script removing all links, rules, and namespaces.	Leaves zero residual state or dangling iptables rules.
<code>proofs.log</code>	Terminal logs & packet capture traces (<code>tcpdump</code>).	Contains ARP resolution, SNAT translation, and DNAT rewrite.
<code>README.md</code>	Engineering post-mortem and packet-flow walkthrough.	Documents lifecycle of an ingress and egress packet.